

Capstone Project Problem Statement

“Smart Health Insurance Management System”

A Modern Java Full-Stack Web Application using Spring Boot & Angular

1. Business Context

A health insurance provider offers multiple insurance plans to individual customers and corporate clients. Currently, the organization faces:

- Manual policy enrollment and renewals
- Limited visibility into customer policies and coverage
- Inefficient claim submission and processing
- Poor coordination between customers, hospitals, and insurance staff
- Limited reporting on claims, premiums, and payouts

The organization wants to build a **secure, scalable, and modern web-based Health Insurance Management System** that:

- Supports multiple user roles
 - Manages policies, members, and claims efficiently
 - Enables end-to-end claim lifecycle management
 - Provides dashboards and analytical reports
 - Works seamlessly on desktop and mobile devices
 - Is cloud-ready and API-driven
-

2. Project Objective

To design and develop a **Full-Stack Enterprise Application** that:

- Uses **Java Spring Boot** as the backend
- Uses **Angular** for frontend development
- Implements **JWT-based authentication and authorization**
- Applies **layered architecture and clean coding practices**

- Demonstrates real-world **CRUD operations, security, validation, and reporting** use cases
-

3. Technology Stack

Backend

- Java 17+
- Spring Boot 3.x
- Spring Web (REST APIs)
- Spring Data JPA
- Hibernate ORM
- PostgreSQL / MySQL
- Spring Security with JWT
- Swagger / OpenAPI
- Maven / Gradle

Frontend

- Angular (latest version)
- TypeScript
- Angular Material / Bootstrap
- Template-Driven / Reactive Forms
- HttpClient & Interceptors

Others

- Postman for API testing
 - Git & GitHub
-

4. User Roles

Admin

- Manage users, roles, insurance plans, hospitals, and policy rules

Insurance Agent

- Enroll customers and manage policy details

Claims Officer

- Review, approve, or reject insurance claims

Hospital / Provider

- Submit treatment details and insurance claims

Policy Holder (Customer)

- View policies, pay premiums, and track claims
-

5. Core Modules & Features

A. User & Security Management

- User registration and login
 - Role-Based Access Control (RBAC)
 - JWT-based authentication
 - Password hashing and validation
 - Secured REST API endpoints
-

B. Policy & Plan Management

- CRUD operations for insurance plans
 - Policy enrollment and renewal
 - Coverage limits and premium calculation
 - Policy status tracking (Active / Expired / Suspended)
-

C. Claims Management

- Submit insurance claims with document upload
- Claim lifecycle:
Submitted → In Review → Approved → Paid / Rejected
- Validate claims against policy coverage rules

- Claim history and status tracking
-

D. Hospital & Provider Management

- Manage network hospitals and providers
 - Link claims to hospitals
 - Track hospital-wise claim statistics
-

E. Billing & Payments

- Premium payment tracking
 - Claim payout tracking
 - Payment status (Pending / Completed)
 - Mock payment gateway integration
-

F. Reporting & Dashboards

- Policies by type and status
 - Claims by status, amount, and hospital
 - Premium vs payout reports
 - High-value claim analysis
-

G. Notifications (Basic)

- Policy enrollment and renewal alerts
 - Claim status update notifications
-

6. Non-Functional Requirements

- RESTful API design
 - Centralized exception handling
 - Secure, scalable, and maintainable codebase
-

7. Backend Architecture Expectations

- **Layered Architecture:**
 - Controllers
 - Services
 - Repositories
 - Models / DTOs
 - DTO usage to avoid exposing entities directly
 - Global exception handling using `@ControllerAdvice`
 - Business logic implemented in the service layer
 - Configuration via `application.yml` / `application.properties`
-

8. Angular Frontend Requirements

- **Modular Architecture:**
 - Core Module – authentication, interceptors, guards
 - Shared Module – reusable components, pipes
 - Feature Modules – Policies, Claims, Admin, Reports
- Template-Driven / Reactive Forms with validation
- JWT HTTP Interceptor
- Route Guards (authentication & role-based access)

Mandatory Pages

- Login / Register
 - Dashboard
 - Insurance Plans
 - My Policies
 - Submit Claim
 - Claims Tracking
 - Reports
 - Admin Panel
-

9. Java & Backend Usage (Mandatory)

Students must demonstrate:

- CRUD operations using Spring Data JPA

- Pagination and sorting
 - JPQL / Criteria API queries
 - Aggregation queries (claims count, payout totals)
 - Validation using `@Valid` and custom validators
-

10. Testing Requirements

- Unit testing using **JUnit 5** and **Mockito**
 - API testing using **Postman**
 - Test cases for authentication and CRUD APIs
 - Environment-based configuration (dev / test)
 - Deployment-ready project structure
-

11. Deliverables

- GitHub repository (Backend + Frontend)
 - Database schema / ER diagram
 - UI screenshots
 - README with setup instructions
 - Final project presentation
-

12. Evaluation Criteria

Area	Weight
Functional completeness	30%
API, Security & JWT implementation	20%
Angular architecture & UI	20%
Backend design & JPA usage	10%
Code quality & error handling	10%
Documentation & demo	10%

